Cryptography

CSUMB

Sam Ogden

OSTEP 56

Updated 2026-05-13 11:07

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Lecture Objectives

- Distinguish confidentiality, integrity, and authenticity (authentication)
- Compare symmetric vs. asymmetric encryption (and why we mix them)
- Explain hashes and digital signatures at a high level
- Recognize common ways systems fail
  - TLDR: “don’t roll your own”

Major Goals of Cryptography

Confidentiality

- Hide the information from prying eyes
- Encryption

Authenticity (Authentication)

- Ensure the data came from who it says it came from
- Signing

Integrity

- Ensure the data is correct
- Cryptographic hashes + MACs (not simple checksums)

Confidentiality

Ensuring data is secret

Two main kinds of Encryption

- Symmetric Key encryption
  - Both the sender and receiver use the same key
  - Examples: DES, AES, Blowfish
- Asymmetric key encryption
  - The sender and the receiver use two different keys, the public key and a private key
  - Examples: RSA, ECC

Shared-Key Encryption (Symmetric)

We are given:

- An algorithm (aka `cipher`), E
- Some data (aka `plaintext`), P
- One secret (aka `key`), K

$$C = E(P, K)$$
$$P = D(C, K)$$

We output:

- Encrypted data (aka `ciphertext`), C

Public-Key Encryption (Asymmetric)

We are given:

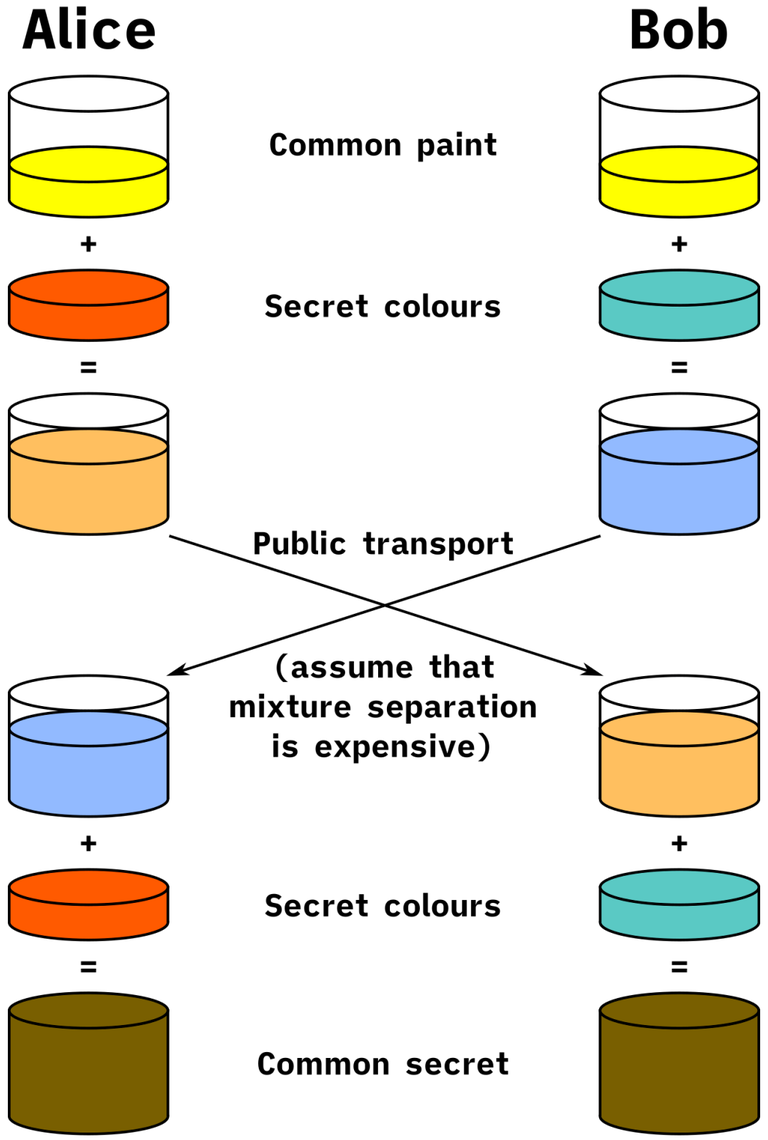
- An algorithm (aka `cipher`), E
- Some data (aka `plaintext`), P
- Two keys
  - A **public** K_{public}
  - A **secret** $K_{private}$
  - The scheme is public; the keys are what matter

$$C = E(P, K_{public})$$
$$P = D(C, K_{private})$$

Trade-offs

- Public-key encryption is expensive
  - Long keys (thousands of bits)
  - Lots of calculations (huge exponents)
- Shared Key encryption is onerous
  - Either you have many unique keys, or you reduce privacy greatly
- Often we use a mix of both!

Diffie-Hellman Key Exchange



Goal: agree on a shared secret, then switch to fast symmetric encryption.

Encryption Summary

- Encryption protects confidentiality
- The algorithm is public; security comes from the key, not from hiding the method
- Symmetric encryption uses one shared secret key
- Asymmetric encryption uses a public key to encrypt and a private key to decrypt
- Good systems use encryption as part of a larger design, not as a magic shield

Integrity

Verifying data is right

Integrity: Core Idea

- Include a piece of information to ensure that a file is unaltered
- We often accomplish this by adding an extra piece of information: a cryptographic hash
- Hashes are one-way functions that represent data in a compact digest

Features of Cryptographic Hashes

- **Collision Resistance**: Collisions are computationally infeasible
- **Input sensitivity (avalanche)**: flipping 1 input bit should flip each output bit with probability ~1/2
  - So on average ~50% of output bits change (with normal variation)
  - Systematic deviation from 50% can indicate bias / weak diffusion
- **One-way**: Computationally infeasible to infer input properties

Computationally Infeasible

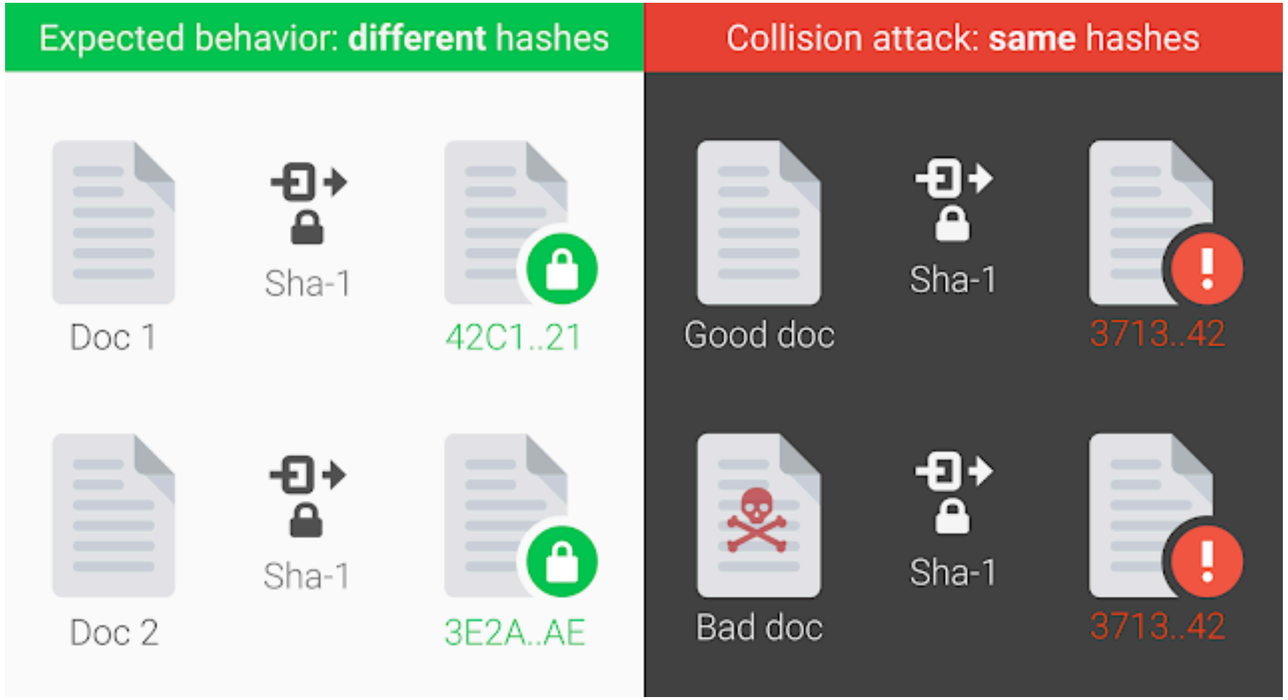
TIME IT TAKES A HACKER TO BRUTE FORCE YOUR PASSWORD IN 2023

Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	Instantly	Instantly	Instantly
6	Instantly	Instantly	Instantly	Instantly	Instantly
7	Instantly	Instantly	1 sec	2 secs	4 secs
8	Instantly	Instantly	28 secs	2 mins	5 mins
9	Instantly	3 secs	24 mins	2 hours	6 hours
10	Instantly	1 min	21 hours	5 days	2 weeks
11	Instantly	32 mins	1 month	10 months	3 years
12	1 sec	14 hours	6 years	53 years	226 years
13	5 secs	2 weeks	332 years	3k years	15k years
14	52 secs	1 year	17k years	202k years	1m years
15	9 mins	27 years	898k years	12m years	77m years
16	1 hour	713 years	46m years	779m years	5bn years
17	14 hours	18k years	2bn years	48bn years	380bn years
18	6 days	481k years	126bn years	2tn years	26tn years

HIVE SYSTEMS

› Learn how we made this table at hivesystems.io/password

Hash Collisions: SHA-1



- A **collision** means two different inputs have the same hash output
- SHA-1 collisions were demonstrated publicly in **2017** (Google)
- Takeaway: “collision resistance” is a real requirement, and older hashes break
- Numbers (roughly):
  - ~9 quintillion SHA-1 computations total
  - Thousands of CPU-years + hundreds of GPU-years

Hashes: Small Changes to Big Changes

```
$ sha1 1 2
SHA1 (1) = 356a192b7913b04c54574d18c28d46e6395428ab
SHA1 (2) = da4b9237bacccdf19c0760cab7aec4a8359010b0
```

We change one bit in the input and it changes everything in the output

Hashes: One-way

```
❯ wc -c 56-cryptography.qmd && sha1 56-cryptography.qmd
    4649 56-cryptography.qmd
SHA1 (56-cryptography.qmd) = fb1c5078ca0db9333dd868a4c7ae22a5eabdfe33
```

A hash compresses a large input into a fixed-size digest

It should be infeasible to recover the input from that digest

Hashes vs. attacker-proof integrity

- A plain hash detects accidental corruption
  - But an attacker can modify the file **and** recompute the hash
- To detect malicious tampering you usually need a secret or a key:
  - `HMAC(key, message)` (shared secret)
  - Digital signature (public-key)

Authentication

Verifying we are us

Authentication: two common models

- Shared-secret authentication uses one key on both sides
  - Example: HMAC
- Public-key authentication uses a private key to sign and a public key to verify
  - Example: digital signatures
- Both prove authenticity, but they trust different things

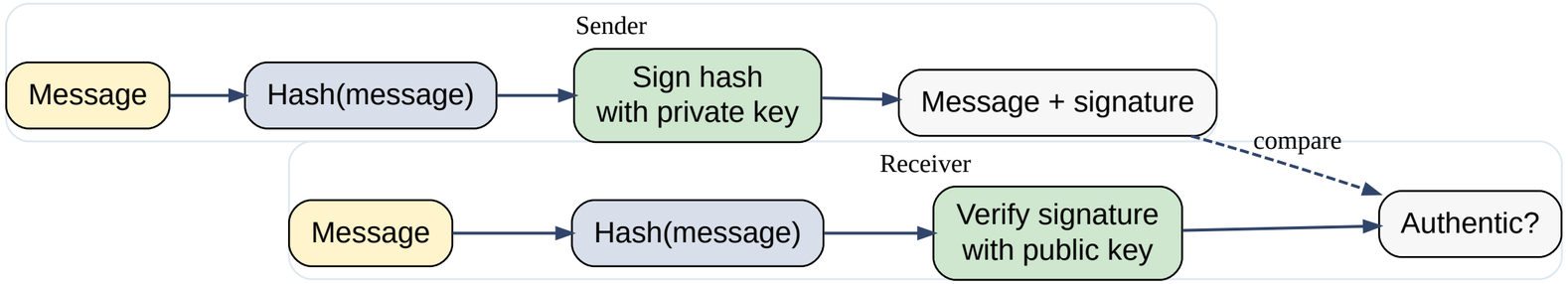
Core Idea: Authentication

Our goal is to prove that we are us, and that we did a thing

- We can do this by sharing something **only** we would know
- We can sign the file with our **private** key, which is known only to us
- Recipient can then verify with our **public** key
  - Remember that our public and private keys are two sides of the same algorithm

Example of Authentication: Digital Signatures

Digital signatures are the public-key version of authentication.



- The sender signs a hash of the message with a private key
- The receiver hashes the message again and verifies the signature with the public key
- If the verification passes, the message is authentic and unchanged

Breaking Cryptography

Common Ways to crack Cryptography

Computational threats

- Brute-forcing keys
- Side-channel attacks
- Quantum Computers

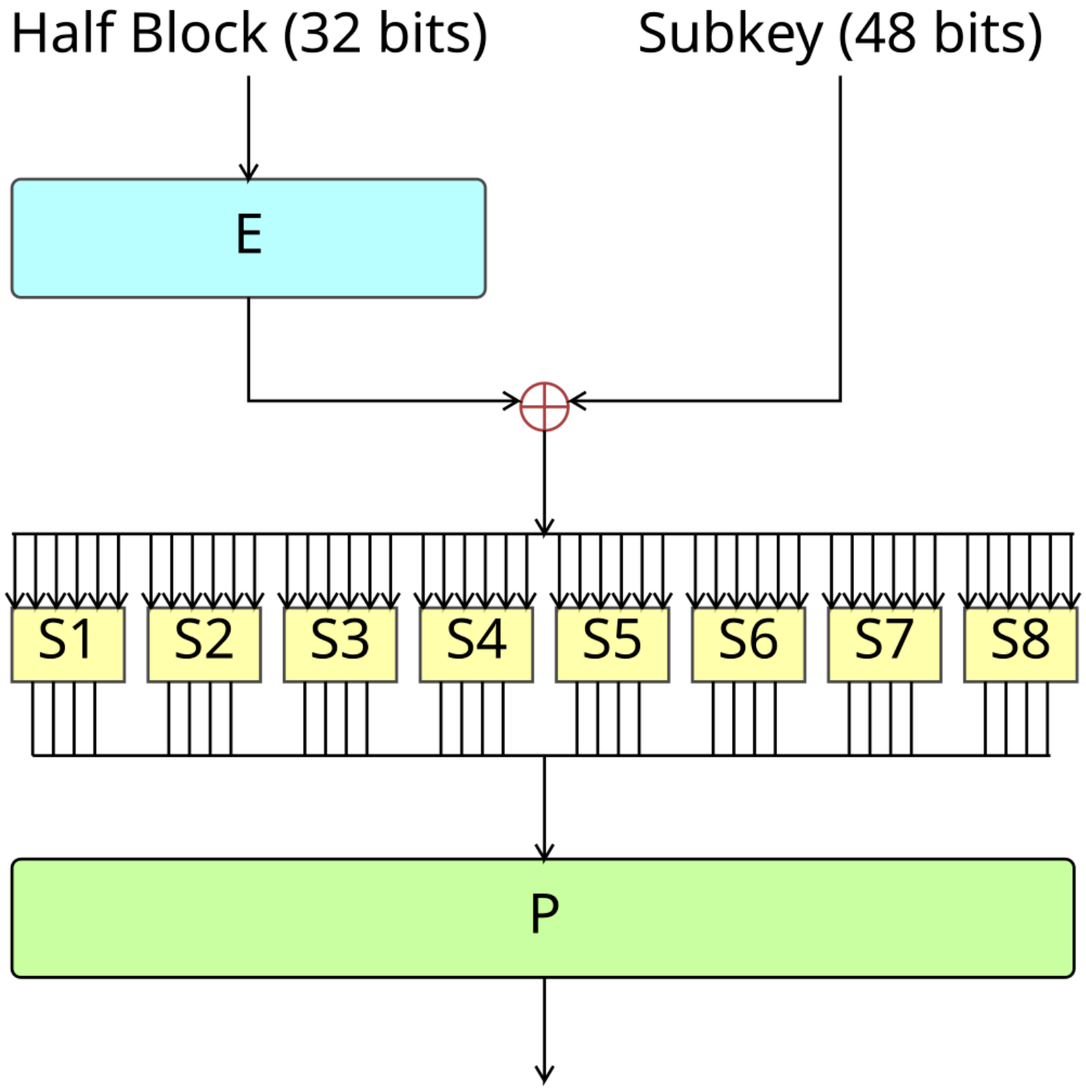
Operational Issues

- Social Engineering
- Backdoors

Design Flaws

- Software errors
- Bad cryptosystem design

Example of a flawed system: DES



Cryptography in Operating Systems

What the OS should think about

- How to provide **entropy**
- How to protect secrets from **other users / attackers**
- How to provide **at-rest data encryption**
- How to safely authenticate users across systems

Conclusion

- Crypto tools map to goals: confidentiality (encryption), integrity (MAC/signatures), authenticity (HMAC/signatures)
- We mix asymmetric + symmetric because performance and key distribution both matter
- Most real-world breaks are operational or implementation failures, not “math is wrong”